Performance Analysis of OpenMP Runtime Parameters for Parallel Pi Approximation

Anwith Goud Ganamaina
Department of Computer Science
Texas State University
San Marcos, TX, USA

***Abstract*—This project analyzes the performance of an OpenMP parallel program that approximates the value of Pi using numerical integration. Experiments were conducted on the Lonestar6 high performance computing system to evaluate the effect of different OpenMP runtime parameters including thread placement, number of threads, and scheduling policies. The results demonstrate that proper thread placement and scheduling strategies significantly influence program performance.**

I. Introduction

Parallel computing allows computational tasks to be executed simultaneously using multiple processor cores. OpenMP is a shared-memory parallel programming model that enables developers to parallelize applications using compiler directives. In this assignment, a serial program that approximates the value of Pi was parallelized using OpenMP. The objective of this project was to study how OpenMP runtime parameters influence performance on the Lonestar6 supercomputer.

II. Task 1: System Architecture using hwloc

The architecture of the Lonestar6 compute node was analyzed using the hwloc tool. The hwloc utility provides detailed information about the hardware topology including sockets, cores, cache levels, and memory.

Ideally the command used is:

```
lstopo --no-io
```

This command reveals the hierarchical structure of the compute node. The system consists of multiple CPU sockets with several cores per socket. Each core has private L1 and L2 caches while an L3 cache is shared among cores within a socket. The node also provides a large shared memory space available to all threads. Understanding this architecture is important when selecting OpenMP thread placement policies such as OMP_PLACES and OMP_PROC_BIND.

III. OpenMP Implementation

The serial program was parallelized using the following OpenMP directive:

```
#pragma omp parallel for reduction(+:sum) schedule(runtime)
```

The parallel for directive distributes loop iterations across multiple threads. The reduction clause ensures that each thread maintains a private copy of the variable sum and safely combines partial results at the end of execution. The schedule(runtime) clause allows scheduling policies to be modified using environment variables without recompiling the program.

TABLE I
OpenMP Variable Scope

Variable	Scope
sum	reduction variable
i	private
x	private
step	shared
num_steps	shared

IV. Thread Placement Performance

Different thread placement configurations were tested using the OpenMP environment variables OMP_PLACES and OMP_PROC_BIND. The results are shown in Table II.

TABLE II
Thread Placement Results

OMP_PLACES	OMP_PROC_BIND	Time (s)
core	close	0.003335
core	spread	0.002798
socket	close	0.002630
socket	spread	0.002743

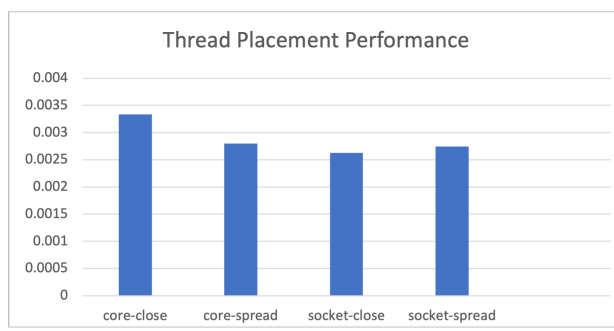

Fig. 1. Thread placement performance comparison

V. Thread Scaling Performance

The program was executed using different numbers of threads to observe how performance scales with increasing parallelism.

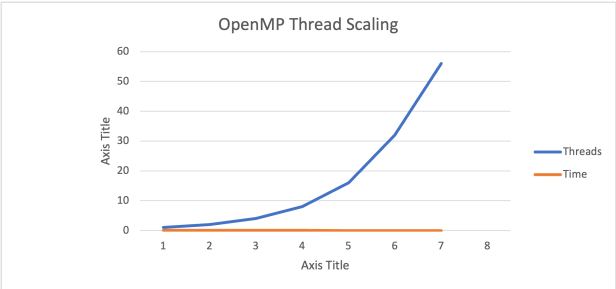

Fig. 2. OpenMP thread scaling performance

The results show that execution time decreases as the number of threads increases because the workload is distributed among multiple processor cores. However, the improvement becomes smaller as the number of threads approaches the maximum number of cores available on the node.

VI. Scheduling Policy Performance

Different scheduling policies were tested to determine how OpenMP distributes loop iterations across threads.

TABLE III
Scheduling Policy Results

Schedule Policy	Time (s)
static,10	0.001291
static,100	0.001236
static,1000	0.001261
dynamic,10	0.029048
dynamic,100	0.003799
dynamic,1000	0.001623

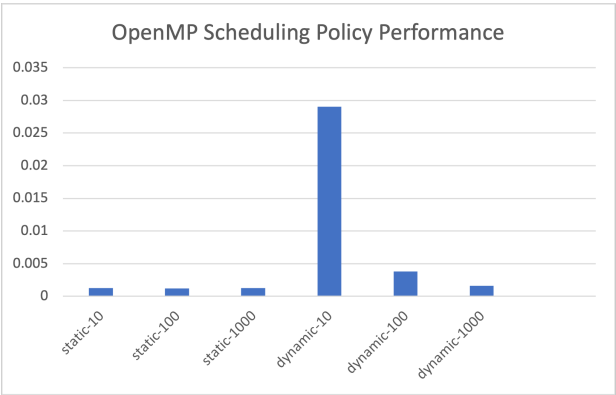

Fig. 3. Scheduling policy performance comparison

VII. Conclusion

This project demonstrated how OpenMP runtime parameters affect the performance of parallel programs on high performance computing systems. Experiments showed that thread placement, number of threads, and scheduling policies significantly influence execution time. The best performance was achieved when threads were placed across sockets with close binding and when static scheduling with moderate chunk sizes was used. These results highlight the importance of understanding hardware architecture and runtime configuration when optimizing parallel applications.